

Exploiting a Vulnerable Web Application

A Home Lab Penetration Testing Write-Up

Date: Sample Write-Up
Environment: Isolated home lab (VirtualBox / TryHackMe-style VM)
Skill Level: Beginner
Category: Web Application Security

Overview

This write-up documents a full penetration test performed against a deliberately vulnerable web application hosted entirely inside an isolated home lab. The goal of this project was to practice the core stages of a web application assessment — reconnaissance, enumeration, exploitation, privilege escalation, and reporting — against a legal, purpose-built target rather than any real-world system.

All testing shown here was performed in a fully isolated, self-hosted lab environment with no internet-facing exposure, in line with responsible and ethical practice.

Objective

- Identify exposed services and enumerate the target application.
- Discover and exploit a vulnerability to gain an initial foothold.
- Escalate privileges to gain full administrative access.
- Document each step clearly enough for someone else to follow and learn from.

Lab Environment Setup

The target machine was deployed as a virtual machine on an isolated internal network, with no bridged or internet-facing adapters. The attacking machine was a separate VM running a standard penetration testing distribution.

Component	Details
Attacker Machine	Kali Linux (VM), internal network only
Target Machine	Custom vulnerable web app VM

Network Mode	Host-only / isolated virtual network
Scope	Target VM only — no external systems in scope

Step 1 — Reconnaissance

The first step was identifying what was actually running on the target. An Nmap scan was used to enumerate open ports and fingerprint running services.

```
nmap -sC -sV -p- 10.10.10.15
```

The scan revealed two open ports: SSH (22) and an HTTP service (80) running an outdated version of a lightweight PHP application, which became the initial focus of the assessment.

Step 2 — Enumeration

With a web service identified, the next step was mapping out the application's structure. A directory brute-force scan was run to uncover hidden pages and endpoints not linked from the main site.

```
gobuster dir -u http://10.10.10.15 -w common.txt -x php
```

This uncovered an admin login page and an upload endpoint that wasn't linked anywhere in the visible site navigation — both flagged as points of interest for the next phase.

Step 3 — Exploitation

Testing the upload endpoint showed it accepted files without properly validating their type or content, which is a classic unrestricted file upload weakness. This made it possible to upload a script disguised as an image and have the server execute it.

After uploading the file and triggering it via its stored path, a reverse shell connection was received on the attacking machine, granting initial low-privilege access to the target.

```
nc -lvnp 4444
```

Step 4 — Privilege Escalation

With an initial foothold established, the next goal was escalating from a low-privileged web-service account to full administrative access. A standard enumeration script was run to check for common misconfigurations.

The scan flagged a scheduled task running on a timer with write access granted to the low-privilege account — a classic path to privilege escalation. By replacing the script the task executed, it was possible to have it run as the higher-privileged account the next time it triggered.

Once the scheduled task ran, a shell was received with full administrative privileges, completing the escalation path.

Key Findings

- **Unrestricted file upload** — the upload endpoint did not validate file type or content, allowing script execution.
- **Hidden endpoints discoverable via brute force** — sensitive pages were reachable despite not being linked publicly.
- **Insecure scheduled task permissions** — a low-privileged account had write access to a script executed by a higher-privileged task.

Remediation Recommendations

- Validate uploaded file types server-side (not just by file extension) and store uploads outside the web root.
- Apply least-privilege file permissions so scheduled tasks can't be modified by lower-privileged accounts.
- Avoid relying on "hidden" unlinked pages as a security control — enforce proper authentication on sensitive endpoints.

Tools Used

Tool	Purpose
Nmap	Port scanning and service fingerprinting
Gobuster	Directory and endpoint enumeration
Netcat	Receiving the reverse shell connection

Custom enum script	Local privilege escalation checks
--------------------	-----------------------------------

Conclusion

This project walked through a complete attack chain against a deliberately vulnerable target, from initial reconnaissance through to full privilege escalation. It reinforced how small misconfigurations — an unvalidated upload field, an overly permissive scheduled task — can chain together into full system compromise, and why defense-in-depth matters at every layer of an application.

This assessment was carried out entirely within an isolated, self-hosted lab environment for educational purposes. No real-world systems were accessed or tested at any point.

This is a placeholder sample write-up to test the PDF-based project format. Replace this content with your own project details and findings.